

How a Promise Behaves

States, settling, await, and why a try/catch only sees what is awaited inside it

A **Promise is a ticket for a value that arrives later**. Calling an async function (or `fetch`, `db.query`, anything from the Promise list) starts the work and hands you the ticket immediately. The function that handed it over is finished; the work continues in the background.

Three states, one transition. A Promise starts *pending* and later *settles* exactly once, into one of two outcomes. It never goes back.

State	Meaning	What <code>await</code> does with it
pending	work not finished	pauses the current function until it settles
fulfilled	finished with a value	gives you the value
rejected	finished with an error	throws that error at the <code>await</code> line

“Settles” means fulfilled OR rejected. A pending Promise has not failed; it has not done anything yet.

`await` is the only thing that opens the ticket. `await p` means: stop this function here, keep its place, let other requests run; when `p` settles, resume: if fulfilled, hand back the value; if rejected, throw the error on this exact line. Without `await`, `p` is just an object you are holding, and nothing you do to it (`p.name`, `p + 1`) touches the value inside. (`.then(f)` is the older way to open it; same idea.)

Being a Promise and being awaited are separate facts. `fetch()` returns a Promise whether or not anyone waits. Nothing forces a caller to `await`; it can walk away, or hand the Promise onward with `return`. The Promise still runs and still settles; the question is whether anyone is listening when it does.

`catch` runs only if an error is thrown while execution is still inside the `try` block. A rejection is not a throw. It becomes a throw at the moment something `await`s it. So the `try` covers a Promise only if the `await` on that Promise is inside the block, before the block ends. If the block exits first, that `try` is over; it does not wait around.

no await	with await
t=0 enter try	t=0 enter try
t=0 fetch() starts; returns pending Promise	t=0 fetch() starts; returns pending Promise
t=0 return -> exit try; nothing thrown; catch skipped; function over	t=0 await -> PAUSE here, still inside try
t=300 network fails -> Promise rejects -> nobody awaiting inside any try -> unhandled rejection	t=300 network fails -> Promise rejects
	t=300 the await throws, right here -> catch runs

The `await` keeps the block open across the waiting time. Without it the block is over in microseconds, long before the Promise has anything to say.

The error lives wherever the `await` is. A Promise travels unopened through any number of functions and `try` blocks until something awaits it; that is the `try` whose `catch` runs.

<pre>function getUser(id: string) { try { return fetch(`/users/\${id}`); } catch (e) { log(e); } }</pre>	<pre>// no await inside: this catch can NEVER run // dead code</pre>
<pre>async function main() { try { const res = await getUser("42"); } catch (e) { log(e); } }</pre>	<pre>// the await is inside THIS try // this catch runs on a network failure</pre>

The same rule in every shape. The test is mechanical: is there an `await` on that Promise, inside this `try` block, before the block ends?

Inside a <code>try</code>	Covered?	Why
<code>save(data);</code>	no	never awaited; rejection escapes
<code>await save(data);</code>	yes	thrown at the await, inside the block
<code>return save(data);</code>	no	the Promise leaves the block pending
<code>return await save(data);</code>	yes	waits inside, then returns the value
<code>const p = save(data); other();</code>	no	<code>p</code> never awaited before the block ends
<code>const p = save(data); other(); await p;</code>	yes	awaited before the block ends
<code>items.forEach(async (i) => { await save(i) })</code>	no	the awaits are inside callbacks <code>forEach</code> does not wait for
<code>for (const i of items) await save(i);</code>	yes	each await is inside the block
<code>await Promise.all(items.map(save))</code>	yes	one await covers all of them; first rejection throws

What a rejection does when nobody awaits it. It becomes an *unhandled rejection*: Node prints a warning and, by default in modern versions, crashes the process; a browser logs it to the console. Either way, no `catch` in your code ran, and no cleanup happened.

The fixes, when a `try` encloses no `await`.

Fix	Code	Use when
A: handle here	make the function <code>async</code> ; <code>return await fetch(...)</code> ; after <code>log(e)</code> , <code>throw e</code> or return a fallback	the function should log, add context, retry, or default
B: handle at the caller	delete the <code>try</code> ; keep <code>return fetch(...)</code> ; the caller writes <code>try { await getUser(id) } catch (e) {...}</code>	the function is a pure pass-through

Two things `await` inside a `try` still does not cover. A `fetch` that gets a 404 or 500 *fulfils*, with `res.ok === false`; the `catch` sees only network failures, so add `if (!res.ok) throw new Error(...)` after the await. And a `catch (e)` receives `unknown`; read `e.message` only after `if (e instanceof Error)`.

The review sentence. “The `try` on line *N* encloses no `await`; the Promise leaves the block pending, so the `catch` can never run. Either `async` + `return await`, or drop the `try` and let the caller handle it.”

Say it to yourself for every `try` you meet: “What is awaited inside this block? If nothing, this catch is dead.”